

Meteor: High-Performance Control Message Delivery for Large-Scale Clouds

Gongming Zhao
University of Science and Technology
of China
Hefei, China
gmzhao@ustc.edu.cn

Baoqing Wang
University of Science and Technology
of China
Hefei, China
bbq@mail.ustc.edu.cn

Min Chen
Huawei Technologies
Hangzhou, China
chemmin148@huawei.com

Hongli Xu*
University of Science and Technology
of China
Hefei, China
xuhongli@ustc.edu.cn

Jiawei Liu
University of Science and Technology
of China
Hefei, China
liujiawei@mail.ustc.edu.cn

Xuwei Yang
Huawei Technologies
Beijing, China
yangxuwei1@huawei.com

Liguang Xie
Independent
Beijing, China
Liguang.vt@gmail.com

Yongqiang Yang
Huawei Technologies
Beijing, China
yangyongqiang@huawei.com

Ying Xiong
Huawei Technologies
Beijing, China
ying.xiong2@huawei.com

Abstract

Virtual private clouds (VPCs) play a critical role in providing secure and isolated network environments for web services. However, with the growing number and size of VPCs, efficiently delivering control messages from the control plane to the data plane has become a major concern for cloud vendors. Existing end-to-end transmission solutions (e.g., RPC) will result in substantial overhead in the control plane, while message-oriented middleware-based solutions (e.g., message queue) will lead to high data plane overhead. To address this issue, we design Meteor, a high-performance control message delivery system for large-scale clouds. Specifically, Meteor combines an RPC path with a message queue (MQ) path and employs an auto dual-path switching mechanism to minimize the message delivery latency. Additionally, we propose a VPC-based message delivery and filtering scheme for the MQ path to reduce data plane overhead. We also design a delivery robustness guarantee mechanism to ensure the reachability and consistency of control messages. Meteor has been thoroughly tested with up to 100k container instances. Evaluation results show that Meteor decreases the message delivery latency by 48.8% and reduces the overhead by about 50% in real-world scenarios, compared with state-of-the-art solutions.

CCS Concepts

• **Networks** → **Cloud computing**; **Network management**; • **Software and its engineering** → **Message oriented middleware**.

*Hongli Xu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792135>

Keywords

Cloud Network; Virtual Private Cloud; Control Message Delivery

ACM Reference Format:

Gongming Zhao, Baoqing Wang, Min Chen, Hongli Xu, Jiawei Liu, Xuwei Yang, Liguang Xie, Yongqiang Yang, and Ying Xiong. 2026. Meteor: High-Performance Control Message Delivery for Large-Scale Clouds. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792135>

1 Introduction

In multi-tenant clouds, tenants typically create virtual private clouds (VPCs) to provide logically isolated and customizable network environments for their applications, such as web services. This enables tenants to manage the network within a VPC (e.g., configuring IPs, security groups, and routing policies) [19]. The control plane responds to tenants' management commands by sending control messages, e.g., network configuration messages, to the data plane [1]. Since control messages indicate tenants' configuration intent for VPCs, the fast delivery of these messages is paramount [23, 35, 41]. For instance, cloud vendors aim to launch a VPC with ready network connectivity within 1 second [51]. However, the growing scale of VPCs (e.g., AWS supporting up to 256k instances per VPC [12]) results in a significant surge in control messages (e.g., routing configuration), posing a substantial challenge to the network convergence rate. Thus, enhancing control message delivery performance is increasingly crucial.

There are two typical models for control message delivery in clouds. One is *end-to-end transmission model*, which establishes TCP/UDP connections between the control plane and the data plane (or compute nodes) to reduce transmission latency. Typical end-to-end transmission models include remote procedure call (RPC) [10], message passing interfaces (MPI) [17], and Java Message Service (JMS) [45]. However, establishing TCP/UDP connection for each control message will consume a amount of resources (e.g., CPU)

[24]. Due to the large number of control messages, this model may overload the control plane and greatly increase delivery latency. For example, FaSST [24] shows that when the control plane is under high load, doubling the number of control messages will lead to a 3× increase in delivery latency. The other is *message-oriented middleware (MOM) model* [9], which is built on the concept of the publish/subscribe mechanism (see §2.1 for details) [39]. In this way, if a tenant expects to set the security group policy in a VPC with 1k Virtual Machines (VMs), using MOM-based solutions, the control plane only needs to publish the configuration message once to corresponding message queue. Then, 1k VMs within the VPC will subscribe the message queue to obtain the message. Thus, MOM-based solutions can achieve low control overhead.

However, existing MOM-based solutions also face several significant challenges for control message delivery. Firstly, due to the limited resources on the MOM, a message server can only support a limited number of queues (e.g., a few hundred for Kafka [14]), which is much less than the number of compute nodes (e.g., more than 100k). As a result, each queue should be responsible for delivering control messages to multiple compute nodes (i.e., node-grouping based MOM model [32]). In this case, the compute nodes will receive redundant messages, resulting in high message handling overhead (i.e., high data plane overhead). Secondly, the inherent overhead, such as setting up and maintaining the MQ infrastructure, and the additional communication delay induced by the publish/subscribe mechanism [37], may result in higher delivery latency compared to end-to-end transmission when the controller's resources are sufficient. Thirdly, when encountering the single point of failure of an MQ server, it takes about 50s to restart as evaluated in §??, which is unacceptable for the system robustness. Thus, typical MOM-based solutions such as RabbitMQ [14] and Kafka [20] usually replicate the messages across multiple MQ servers to achieve disaster recovery strategies [53]. However, it will consume double MQ resources.

In summary, we observe that end-to-end transmission can achieve low transmission delay, but may encounter control plane overload. In contrast, MOM-based solutions can keep low control overhead through the publish/subscribe mechanism, but may incur high data plane overhead and high delivery latency. One natural idea is to combine the advantages of the two solutions and conquer their disadvantages. Motivated by this insight, we first conduct an experimental analysis and then propose Meteor to achieve high-performance control message delivery. The key ideas of Meteor are threefold: (1) To achieve low-latency control message delivery in various scenarios, we design a dual-path (RPC and MQ) delivery mechanism, which can select delivering path according to the characteristics and requirements of control messages. (2) In order to achieve low overhead in both control plane and data plane, we propose VPC-based message delivery and filtering mechanism for MQ path to enhance today's node-grouping based MOM model. (3) To ensure the reachability and consistency of control messages, We further design a delivery robustness guarantee mechanism to improve single-point failure recovery capability. The main contributions of this paper are as follows:

- We emphasize the importance of optimizing control message delivery, analyze the limitations of several existing typical

control messages delivery solutions, and propose a feasible direction for addressing these challenges.

- We design Meteor, a high-performance control message delivery system, which mainly consists of three mechanisms: auto dual-path switching, VPC-based message delivery and filtering, and delivery robustness guarantee.
- We evaluate the performance on a testbed with a scale of 100k simulated compute nodes. Evaluation results show that Meteor decreases the message delivery latency by 48.8% and reduces the overhead by about 50% compared with state-of-the-art solutions.

2 Background and Motivation

2.1 Limitations of Prior Works

Control messages are used to push user configurations and program flow entries from the control plane to the data plane [13] [38]. There are two types of control messages: *unicast message*, which configure specific VM instances for a tenant (e.g., setting IP addresses or adding/removing ports), and *multicast message*, which apply to multiple instances (e.g., configuring subnets or security group policies). The following are two typical control message delivery models:

End-to-end transmission model enables direct communication between the control and data planes, as illustrated in Figure 1(a). For example, Google's Orion controller [15] employs an RPC-based mechanism, where the controller maintains TCP connections with each compute node to deliver control messages. Although this model achieves low delivery latency, it imposes high CPU overhead on the control plane to sustain numerous end-to-end connections, thereby limiting scalability (e.g., Aquila supports only about 500 nodes per controller [18]). To address this issue, cloud vendors such as Google [27] and Intel [25] deploy distributed controller clusters. However, this approach merely alleviates control-plane overload by resource scaling, without fundamentally reducing the resource consumption caused by large-scale control message delivery.

Message-oriented middleware (MOM) model [9] enables asynchronous communication between the control and data planes via the publish/subscribe mechanism [14, 20, 36], as illustrated in Figure 1(b). In this model, the control plane publishes messages to a message queue (MQ), and compute nodes receive them through subscriptions. For example, if 100 VMs share the same subnet configuration, the control plane only needs to publish the message once, and all VMs can receive it by subscribing to the corresponding topic. This effectively alleviates control-plane overhead and improves scalability. Therefore, the MOM model has been widely adopted by cloud platforms such as IBM Cloud [6], China Unicom Cloud [16], and OpenStack [44]. However, an MQ server can host only a limited number of queues (as described in §1). As the number of compute nodes grows, multiple nodes must share the same queue. Existing solutions group nodes by physical attributes (e.g., IP address) [4, 32], causing each node to receive redundant messages and leading to increased data-plane overhead.

2.2 Preliminary Experiment

We set up a preliminary experiment to test the latency of the end-to-end transmission solution and the MQ based solution when delivering unicast and multicast messages to different numbers of

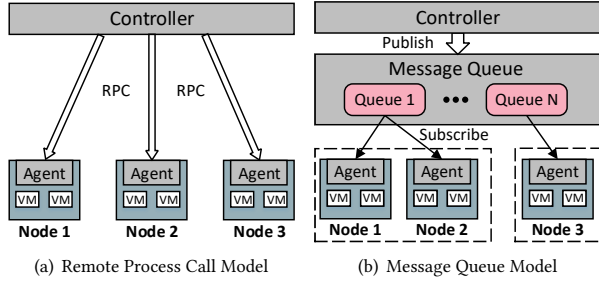


Figure 1: Two Typical Control Message Delivery Models. (a) RPC provides an end-to-end transmission capacity over TCP/UDP connections. (b) MQ delivers messages by publish/subscribe mechanism.

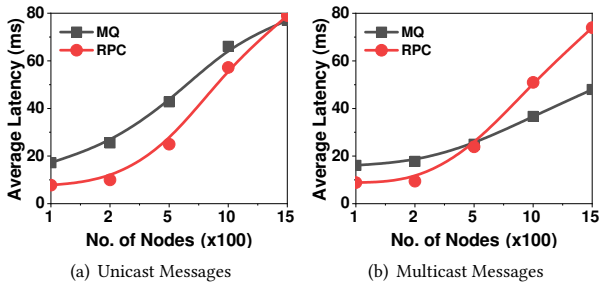


Figure 2: Latency vs. No. of Compute Nodes

compute nodes (e.g., 100-1500). Specifically, we use two servers to build a preliminary testbed and each server is equipped with a 22-core Intel Xeon 6152 processor and 256GB memory. One server is used as the Meteor controller (shared with MQ server), and the other creates processes to simulate compute nodes. The specific implementation details align with those outlined in §7.1.

We let the controller send 2 unicast (multicast) messages to each compute node through the RPC path and MQ path respectively. As shown in Figures 2(a)-2(b), the RPC path is generally suitable for delivering unicast messages while the MQ path is suitable for delivering multicast messages. Moreover, RPC achieves lower latency on a small scale but performs worse as the scale increases. To fully understand the performance differences, we provide a detailed breakdown of latency, as shown in Figures 3(a)-3(b). When the network scale is small (e.g., with 200 compute nodes), we observe that the MQ solution achieves a higher message delivery latency compared to the RPC solution. This is due to the fact that the MQ solution employs the publish/subscribe mechanism to implement asynchronous communications, which incurs additional communication delay in the MQ server. On the contrary, when the network scale is larger (e.g., with 1500 compute nodes), the RPC solution encounters a bottleneck at the controller, resulting in higher message delivery latency compared to the MQ solution, especially in the multicast message delivery scenario.

As summarized in Table 1, end-to-end transmission can achieve low latency and low data plane overhead through providing direct communication between the control and data planes. However, when delivering massive control messages, this model may overload the control plane. On the contrary, by adopting the publish/subscribe mechanism, MOM-based solutions greatly reduce

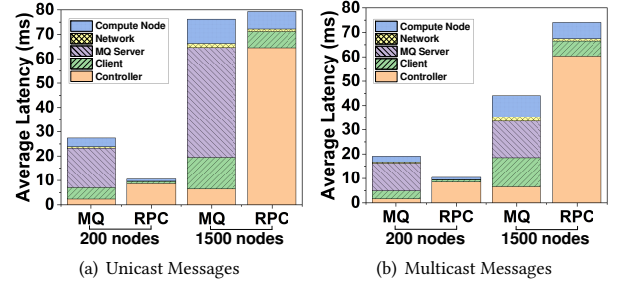


Figure 3: Detailed Breakdown of the Latency

Table 1: Comparison of Typical Models in Clouds

Models	Latency	Control Plane Overhead	Data Plane Overhead
End-to-End Transmission	Low	High	Low
Message Queue	Medium	Low	High
Ours: Meteor	Low	Low	Low

control plane overhead, but at the expense of high data plane overhead and a certain increase in delivery latency.

A natural idea is to combine the advantages of the two solutions and conquer their disadvantages. Motivated by this insight, we aim to design a high-performance control message delivery system for large-scale clouds.

3 System Design

3.1 Design Goals

To address the increasing challenges associated with control message delivery in large-scale clouds, we design Meteor and the design goals are as follows:

- **Low Latency:** Many tenants expect to quickly deliver control messages from control plane to data plane (*i.e.*, low latency) so as to execute commands immediately. Otherwise, it will lead to a worse tenant experience.
- **Low Overhead:** Since overload may affect the system's processing performance and increase delivery latency, Meteor aims to deliver control messages with lower overhead, in both control plane and data plane.
- **High Robustness:** In order to ensure correct execution of commands in clouds, Meteor needs to guarantee the reachability and consistency of control messages.

3.2 System Overview

As shown in Figure 4, we propose Meteor, which consists of three core mechanisms: auto dual-path switching, VPC-based topic subscription and filtering, and delivery robustness guarantee.

Auto Dual-path Switching aims to reduce the message delivery latency by auto-switching to the appropriate delivery path. Switching the message delivery paths between RPC and MQ is enabled by a *path controller* in the control plane. Specifically, when the control plane handles the control messages from the northbound RESTful API, the path controller switches the delivery path to the appropriate one for the current messages (§4).

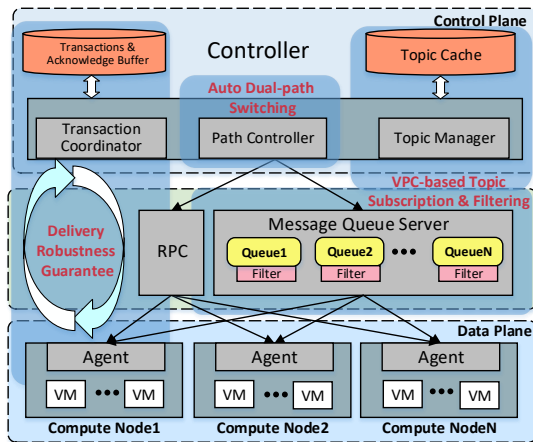


Figure 4: Meteor Overview. Auto dual-path switching mechanism switches the message delivery paths between RPC and MQ to reduce the message delivery latency. VPC-based topic subscription and filtering mechanism reduces the overhead in the process of control message delivery through MQ. Delivery robustness guarantee mechanism ensures the reachability and consistency of control messages.

VPC-based Topic Subscription and Filtering reduces the overhead of control plane and data plane in the process of control message delivery through MQ. Specifically, two modules (*topic cache* and *topic manager*) are established in the control plane to implement VPC-based topic subscription for the MQ path. Additionally, a *message filter* is configured for each queue to avoid message redundancy, thereby reducing the data plane overhead (§5).

Delivery Robustness Guarantee consists of message reachability and consistency guarantee. Specifically, two modules (*transaction coordinator* and *transactions & acknowledgment buffer*) are designed in the control plane to guarantee the reachability of messages. An *agent* module is developed on each compute node to maintain the order of received messages, providing an order-preserving and consistent message delivery service (§6).

4 Auto Dual-path Switching

Meteor first determines the optimal delivery path for a VPC's control messages. At any given time, a VPC selects either the RPC or MQ path for two reasons. First, the large latency gap between the two paths may cause message reordering if used concurrently. Second, making per-message path decisions would incur excessive control overhead due to the high message volume. However, with the continuous change of VPC size, the choice of message path for a VPC should be changed dynamically. Specifically, the size of a VPC can be measured by the number of ports and the number of compute nodes. On the one hand, the total number of control messages for a VPC generally depends on the number of ports (or VMs) in the VPC [11, 48]. On the other hand, the number of multicast control messages received by the data plane depends on the number of compute nodes associated with this VPC [18]. We design a dual-path automatic switching mechanism according to these two dimensions to adapt to changes in VPC size.

Briefly, the small VPCs start with the RPC path and change to MQ path as its size grows. Conversely, the large VPCs will switch the

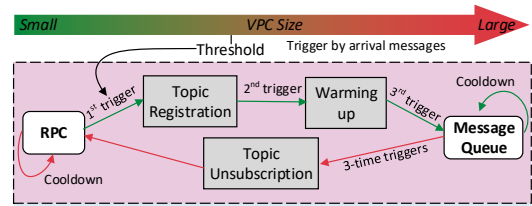


Figure 5: Illustration of dual-path switching workflow. As the growing of VPC size, the message path switches from RPC to MQ after 3-times triggers, including topic registration (after the first trigger) and warming up (after the second trigger). Conversely, it switches the message path from MQ to RPC after 3-times triggers.

message path from MQ to RPC. However, automatic path switching faces several challenges, including how to prevent thrashing caused by frequent switching, how to enable compute nodes to dynamically subscribe/unsubscribe to MQ topics, and how to make new paths take effect quickly. To conquer these challenges, we give the detailed switching process in Figure 5. For simplicity, we set two thresholds, τ_P and τ_N , corresponding to the two dimensions of VPC size. Specifically, τ_P denotes the threshold number of ports and τ_N denotes the threshold number of nodes. These thresholds can be set in two ways. One is artificial parameter setting based on experience, such as the auto scaling policy used by AWS [46] and some previous works [8, 42]. The other is parameter estimation based on system performance. We give a convenient method using queueing theory to achieve the desired delivery latency in Appendix A.

From MQ to RPC: When the control plane needs to send a control message, we check the size of this VPC. If the current number of VPC ports is less than τ_P or that of VPC nodes is less than τ_N , the threshold is said to be triggered once. To avoid thrashing caused by temporary changes of VPC size, the message path of the vpc will be switched from MQ to RPC only after several triggers have been accumulated (e.g., 2-4 times). Meanwhile, the controller updates the topic cache and notifies corresponding compute nodes to unsubscribe. After a VPC’s message path is switched, the controller will wait for a cooldown period (e.g., 5-15 minutes [3, 28]) and forward control messages of the VPC without making a threshold judgment. The design of cooldown period aims to prevent frequent path switching caused by continuous jitter of VPC size. Note that, the number of triggers and cooldown period are set based on VPC properties. According to our experience, using 3-time triggers and a 10-minute cooldown period can achieve a good trade off between path switching overhead and control message transmission efficiency for most VPCs.

From RPC to MQ: With the increasing size of a VPC, it naturally occurs a large number of both unicast and multicast messages, leading to performance degradation of message delivery using RPC. Thus, the path switching from RPC to MQ will be triggered by the current number of VPC ports $\geq \tau_P$ and that of VPC nodes $\geq \tau_N$. Besides multiple triggers and cooldown mechanisms, we give the following design to start MQ path “quickly”. After the first trigger, a topic will be created in MQ server and recorded in the topic cache. After the second trigger, the controller registers the topic in MQ server, which then creates a corresponding queue. Meanwhile, several empty messages will be sent to this topic to warm-up this

queue. After the last trigger, the controller notifies the corresponding compute nodes to subscribe to MQ topic. Additionally, in case the VPC size down after the first or second triggers, *i.e.* the third trigger will not happen, we still keep the topic in the cache and its queue in the MQ server. When another VPC triggers, that VPC will be assigned with this topic instead of creating a new one.

5 VPC-based Topic Subscription and Filtering

5.1 VPC-based Topic Subscription

With the development of virtualization, tenants typically manage VMs through VPCs. Unlike previous node-grouping based solutions [32][44], we introduce a VPC-based topic subscription mechanism. Under the VPC architecture, control messages carry both physical attributes (*e.g.*, destination IP) and virtual attributes (*e.g.*, VPC ID). We find that messages within the same VPC are usually delivered to compute nodes sharing the same virtual address. Hence, assigning MQ topics based on VPC attributes can improve delivery efficiency.

Due to the limited capacity of MQ servers (typically around 20 topics per 3-server cluster [47]), each VPC is assigned a dedicated topic only when the number of VPCs is below this threshold. A topic cache maintains the mapping between VPCs and topics. When a new VPC joins, the topic manager first searches for idle topics (*e.g.*, released from VPCs that switched from MQ to RPC). If none are available, a new topic (named by the VPC ID) is created and recorded in the cache. When topics become insufficient, multiple VPCs are grouped into one topic. To minimize queue usage, we design a greedy topic-assignment algorithm. Additionally, to avoid message interference among VPCs, a tag-based filtering mechanism is introduced. Details of the assignment algorithm and tag design are provided in Appendix B.

5.2 Message Filtering

According to the above design, we find that each node will not receive messages belonging to other VPCs. However, a compute node may still receive the messages which belong to other compute nodes inside this VPC. To this end, we further design a node-based message filtering mechanism to fully eliminate message redundancy. Specifically, each message is assigned a tag indicating its destination: for unicast messages, the tag is the receiver's unique node ID (*e.g.*, IP address); for multicast messages, the tag is a public identifier "#". Moreover, each compute node sets two tags, compute node ID and a public string "#". These tags are stored in the message header and only occupy several bytes. If a message's tag can match with one of two tags of a compute node, we call this message matching with the compute node. Therefore, a message will be delivered to the matched compute node(s).

Since each control message requires tag matching, we employ a hash-based mechanism to accelerate tag lookups. Each compute node maintains two hash values, one corresponding to each tag. When a message is to be delivered, the filter computes the hash value of its tag and forwards the message to the compute node(s) with the matching value. However, the finite capacity of the hash map may cause collisions as the number of compute nodes in a VPC increases. A straightforward mitigation is to adopt conventional collision-handling techniques (*e.g.*, linear probing or double hashing [54, 55]). When the number of nodes approaches the theoretical capacity of the hash map, we further partition the VPC

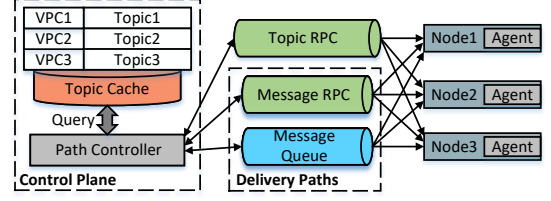


Figure 6: Illustration of dynamic topic subscription. The topic cache stores the correspondence between topics and VPCs. The path controller queries topic information from the cache and notifies the compute nodes to subscribe/unsubscribe to/from the topic through the topic RPC path.

into multiple sub-VPCs, each assigned a distinct topic for message delivery. These sub-topics handle messages within their respective sub-VPCs, preventing performance degradation due to excessive hash collisions.

5.3 Dynamic Topic Subscription

In practice, each compute node will be served by different VPCs with time passing. Consequently, each compute node needs to subscribe to variable topics, *i.e.*, dynamic topic subscription. The subscribed topics of each compute node are determined by associated VPCs with this compute node. That is, if there are one or more VM/container located on a compute node, this node needs to subscribe to the corresponding topic(s). As shown in Figure 6, to facilitate dynamic topic subscription, we design the topic RPC path to deliver subscription information and store the topic mapping information in the memory cache (called topic cache in our system).

When a new VM belonging to a VPC is configured on a compute node, the compute node should update its subscribed topics dynamically. To achieve this, the compute node first identifies the VPC to which the VM belongs. If it has not subscribed to the relevant topics previously, the compute node queries the database to retrieve the corresponding topic(s). Subsequently, it subscribes to this topic and starts receiving messages from the corresponding queue(s). The same process will be performed when a VM is migrated or updated to achieve dynamic topic subscription.

6 Delivery Robustness Guarantee

6.1 Message Reachability Guarantee

Dual-path delivery poses a challenge for at-least-once semantic assurance compared to sending messages through a single path. To this end, a distributed transaction is introduced to the system. To establish the message transaction mechanism, there are three components: transaction coordinator (TC), transaction buffer (TB) and acknowledgments manager (AM). Specifically, TC maintains the entire life cycle of transactions and handles transaction timeout. TB persists all the transaction metadata, and AM maintains message acknowledgments within a transaction before its completion.

The workflow is illustrated in Figure 7 and described as follows. (i) *Transaction registration.* For each batch of message delivery, a transaction (txn) is registered by the TC. One transaction may be responsible for managing one or several message(s). The pending committed transaction will be logged into the TB. (ii) *Ack registration.* The pending message acknowledgments will be registered in AM. The number of message acknowledgments depends on the message path. Specifically, AM will register multiple Acks for RPC path,

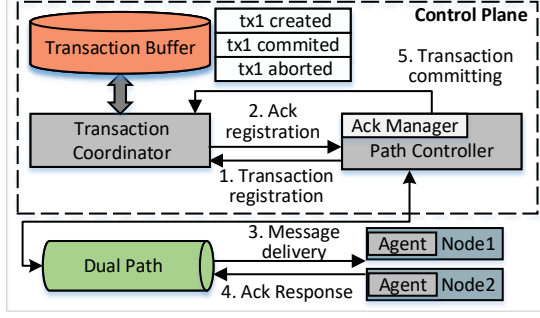


Figure 7: Illustration of message transaction processing. The workflow of transaction messages consists of five steps. (1) A transaction for these messages is registered by transaction coordinator and logged in transaction buffer. (2) The pending acknowledgments are registered in acknowledgment manager. (3) Messages are delivered to the data plane. (4) Acknowledgments are responded to the AM. (5) The transaction is committed or rolled back.

i.e., one for each connection, while only single Ack will be registered for MQ. We note that MQ itself has transactional provisioning of messages. (iii) *Message delivery*. The controller finishes delivering the messages and waits for the acknowledgments. The failed delivery operations are going to be retried in several times (*e.g.*, 3). (iv) *Ack Response*. When agents receive the messages successfully, acknowledgments will be responded to the AM through the corresponding delivery path. (v) *Transaction committing*. When all the acknowledgments are received, the AM commits the transaction to the TC and marks as *Committed*. Otherwise, the AM notifies the TC to roll back this transaction and mark it as *Aborted*. The above message transaction processing ensures message reachability.

6.2 Order-preserving and Consistency Guarantee

We deploy Meteor agent on each compute node to provide a consistent message-receiving service. Meteor agent mainly contains two monitoring programs: RPC server and MQ client. The RPC server is responsible for listening to the port corresponding to the RPC path, so as to receive messages through the RPC path. The MQ client dynamically subscribes to corresponding queue(s) according to the control plane's commands (issued through the RPC path), thereby receiving messages from the specified message queue(s).

Due to network dynamics, message delivery latency along different paths is unpredictable. Consequently, Meteor's dual-path delivery may result in out-of-order messages at compute nodes. Since many control messages are strictly dependent, such reordering can cause severe errors, making order preservation essential. The control message ordering problem in dual-path delivery is analogous to packet ordering in traditional multi-path forwarding. Following prior works [31, 50], the Meteor controller assigns a timestamp to each control message. Each agent maintains a local cache recording the latest timestamp of received messages. Upon receiving a new message, the agent compares its timestamp with the cached value. If the message is outdated, the agent discards it, rolls back all subsequent messages on the node, and reports an out-of-order error to the controller.

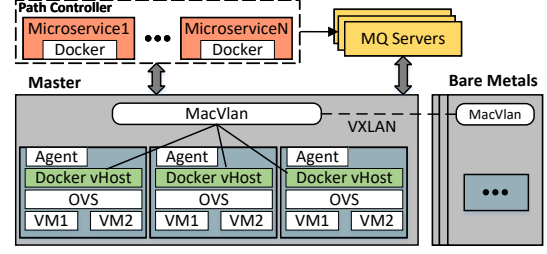


Figure 8: Illustration of the testbed topology.

7 Evaluation

7.1 Experimental Settings

Testbed Setup. We use 24 servers to build a testbed, all running Ubuntu 18.04. Among them, 1 server is used as the Meteor controller, 3 servers are used as the message queue cluster and other 20 servers are as compute nodes. Each server is equipped with dual 22-core Intel Xeon 6161 CPUs, 612 GB memory and a 10GbE NIC. The controller is packaged as Spring Boot service in docker container and the caches are implemented by Apache Ignite [2]. Moreover, the controller and the caches are managed by Kubernetes Cluster [5]. For the data plane, the production environment should be deployed with one agent per physical host. Considering the limited physical resources and the main focus of Meteor is control message delivery, we adopt the following two effective methods to scale up our evaluation.

- (1) **Full Agent:** Similar to [29], we use docker containers to simulate compute nodes. As shown in Figure 8, each container uses Linux network namespaces to emulate virtual VMs. A Meteor agent is deployed in each container and connected via a MacVlan network [7]. Meanwhile, an Open vSwitch instance [40] runs inside each container to establish inter-VM connections through the VXLAN protocol. Each container requires about 1.2GB of memory, allowing roughly 500 containers per physical host (*i.e.*, bare-metal server). We refer to this setup as *Full Agent*, which can scale up to 10k simulated nodes.
- (2) **Lite Agent:** To evaluate Meteor under larger scales, we also implement a lightweight version of the agent by removing the message execution module and focusing solely on message delivery. In this setup, each process simulates a compute node, and a single bare-metal server can host up to 5k processes. We denote this setup as *Lite Agent*, which supports simulations of up to 100k nodes.

Performance Metrics. We give an outline of the following three sets of performance metrics.

- (1) **Latency.** We record the time interval from the controller sending the message to the compute node receiving the message as the message delivery latency. We compute the average delivery latency of all messages during the system running as a metric.
- (2) **Completion Time.** We record the time interval from when the controller starts sending this batch of messages to when the compute nodes receive all the messages as the completion time. Obviously, for the same batch of messages, the shorter completion time means a faster message delivery speed.

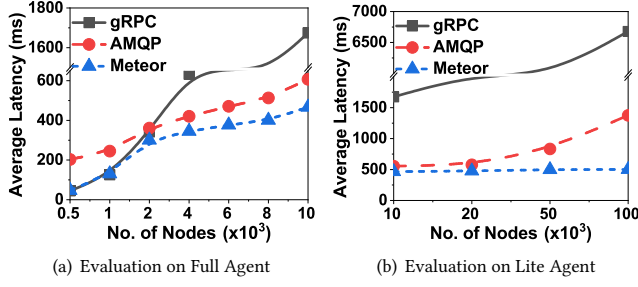


Figure 9: Average Message Latency vs. No. of Nodes

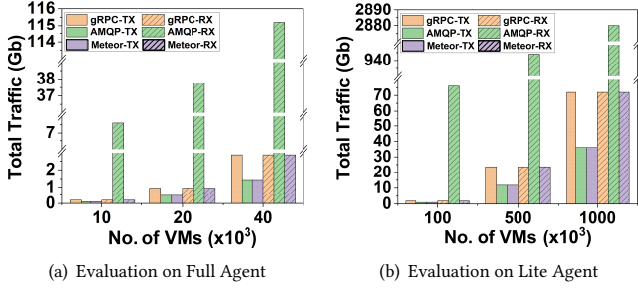


Figure 11: Overhead vs. No. of VMs

- (3) **Overhead.** The control plane overhead is illustrated by the *CPU utilization* of the controller and the data plane overhead is indicated by the *total traffic* received of compute nodes.

Benchmarks. We compare Meteor with two representative benchmarks: (i) gRPC [22], a typical end-to-end transmission model. As described in §2.1, gRPC establishes direct links between the controller and agents in Docker containers. A thread pool of 88 threads (twice the number of CPU cores) is deployed at the controller to run multiple gRPC clients in parallel. Each client uses a bidirectional streaming RPC, enabling concurrent message exchanges between the client and server. (ii) AMQP [49], a representative message-oriented middleware (MOM) model widely used in OpenStack. Given that the number of compute nodes (10k–100k) greatly exceeds the number of queues (100–1k), we adopt a node-grouping strategy in which each queue serves 100 compute nodes. We implement AMQP using Apache Pulsar [47], deploying ZooKeeper for metadata synchronization, Pulsar Broker for message caching, and BookKeeper for persistence. The JVM stack size is set to 80 GB. Additionally, the controller integrates a Pulsar client with 10 message producers [43].

7.2 Overall Performance

7.2.1 Performance Comparison for a Large Scale VPC. According to [12], AWS supports up to 256k VMs per VPC. In a virtual network, a port entity refers to the virtual network card in the virtual machine or container. Each virtual machine or container typically contains 1-2 ports. Considering the growing demand for VPCs from tenants, the VPC size limit we tested is chosen to be 500k, which is almost 2 times the existing maximum size. We compare the overall performance of Meteor against the benchmarks under the scenario of port creation. In particular, for AMQP, every 100 compute nodes subscribe to a queue as a group.

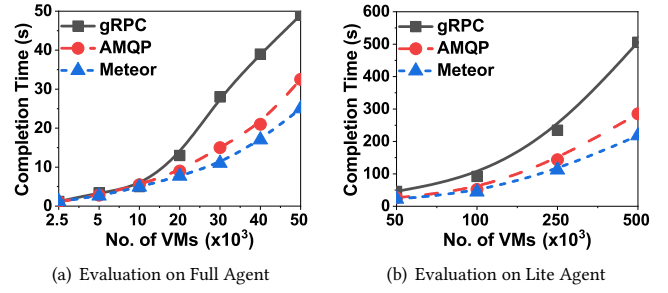


Figure 10: Total Time vs. No. of VMs

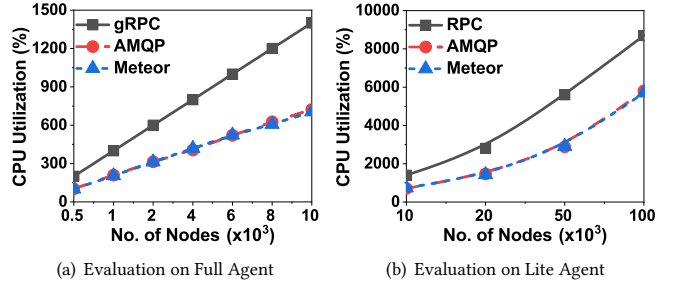


Figure 12: CPU Utilization vs. No. of Nodes

Figures 9(a)–9(b) show that the average message delivery latency of all schemes increases with the number of compute nodes, while Meteor consistently achieves the lowest latency. Specifically, Meteor reduces latency by up to 76% compared with gRPC under 1k nodes and by over 50% compared with AMQP in large-scale VPCs (100k hosts). This improvement benefits from Meteor’s dual-path switching mechanism and VPC-based delivery and filtering design, which together enhance message delivery efficiency.

Second, we evaluate the completion time of delivering a batch of control messages, as shown in Figures 10(a)–10(b). The completion time increases with the number of VMs for all schemes, while Meteor consistently achieves the lowest values. For instance, with 40k VMs, Meteor reduces completion time by 22% and 57% compared with RPC and AMQP, respectively. When the VPC scale is small, Meteor selects the RPC path for lower latency; as the scale grows, its dual-path switching mechanism shifts to the VPC-based MQ path for efficient multicast delivery. Since the MQ enables one-to-many message dissemination in parallel, the controller transmits each message only once, further reducing completion time. Overall, Meteor achieves significantly shorter completion times, especially in large-scale VPCs.

Third, we compare the total traffic received by compute nodes and the CPU utilization of the controller to evaluate the data-plane and control-plane overheads, respectively, as shown in Figures 11(a)–12(b). Figures 11(a)–11(b) show that gRPC incurs the highest control-plane traffic, while AMQP generates the largest data-plane traffic. In contrast, Meteor achieves a balanced overhead—its control-plane traffic is comparable to AMQP, and its data-plane traffic is similar to gRPC. This advantage stems from Meteor’s publish/subscribe mechanism, which mitigates control-plane overhead, and its VPC-based delivery and filtering mechanism, which reduces data-plane redundancy. Moreover, Figures 12(a)–12(b) illustrate that gRPC imposes the highest CPU utilization on the

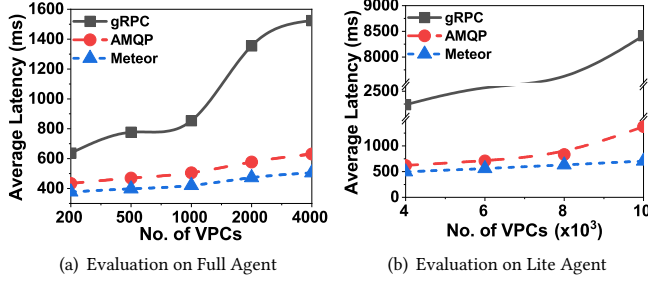


Figure 13: Average Message Latency vs. No. of VPCs

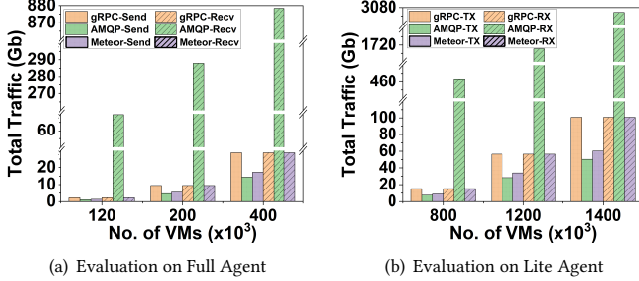


Figure 15: Overhead vs. No. of VMs

controller, whereas AMQP yields the lowest. Meteor shows a moderate CPU load due to partial RPC-path usage, while its dual-path switching mechanism ensures that the control plane remains stable without overload.

7.2.2 Performance Comparison for Multiple VPCs. We further evaluate the performance of Meteor and benchmarks in the multi-VPC scenario. We set the VPC size distribution based on the Huawei Cloud Region A operating data and the Google trace dataset [52]. In these datasets, the VPC size distribution is not uniform. 0.1% of them are VPCs of 10k VM scale, and the rest are VPCs of 3-500 VM scale. Referring to the above VPC scale distribution and considering the growth trend of VPC scale, we set 0.1% of these VPCs as VPCs with a scale of 10k hosts, and the rest of the VPC scales are set to 5-10 hosts (running 5 VMs per host). In the evaluation, when the total number of VPCs is less than 1k, we set it to contain a VPC of 10k host scale. Similar to the setting in Section 7.1, we construct evaluations on 10k agents and 100k lite agents.

First, Figures 13(a)-13(b) illustrate the average message delivery latency by varying the number of compute nodes. As the number of VPCs increases, the message delivery latency for all schemes increases. Since Meteor selects MQ path and RPC path for large and small VPC(s), respectively, it could obtain lower message delivery latency than the other benchmarks in the multi-VPC scenario. For example, when there are 10k VPCs in Figure 13(b), Meteor can reduce the average message delivery latency by about 48.8% and 91.6% compared with AMQP and gRPC, respectively. The above results prove that Meteor can also achieve satisfactory message delivery latency in the multi-VPC scenario.

Second, we evaluate the completion time of delivering a batch of messages by adopting different message delivery schemes and the results are shown in Figures 14(a)-14(b). In general, with the increased number of VMs, the total completion time also increases for all schemes and Meteor always achieves the lowest completion time. For example, when there are 400k VMs in Figure 14(a), Meteor

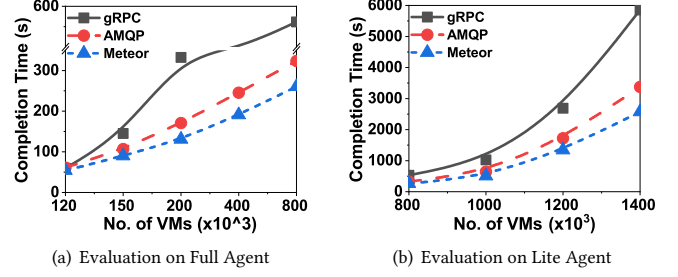


Figure 14: Total Time vs. No. of VMs

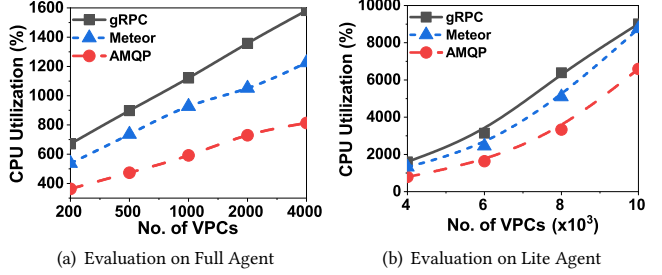


Figure 16: CPU Utilization vs. No. of VPCs

can reduce the completion time by about 22.4% and 58.2% compared with AMQP and gRPC, respectively. The reason is that Meteor can leverage the dual-path switching mechanism to adaptively select the appropriate paths for VPCs of different sizes. This design greatly reduces the IO usage of the controller, so it can achieve higher throughput and reduce completion time.

Third, we also compare the total traffic and the CPU utilization of the controller to show the data plane overhead and the control plane overhead, respectively. As shown in Figures 15(a)-15(b), AMQP generates massive message redundancy on the data plane, and the total traffic received by compute nodes increases rapidly as the size of the VPC increases. In addition, Figures 16(a)-16(b) show that although gRPC can eliminate message redundancy on the host, it brings a higher overhead to the controller. Benefiting from VPC-based messages delivery design and the message filtering mechanisms, Meteor can achieve low overhead on both the data plane and the control plane at the same time.

8 Conclusion

In this paper, we propose Meteor, a high-performance control message delivery system for large-scale clouds. Meteor mainly consists of three core functions: auto dual-path switching, VPC-based message delivery subscription and filtering, and robust delivery guarantee. Comprehensive experiment results show that Meteor decreases the message delivery latency by 48.8% and reduces the overhead by about 50% in real-world scenarios, comparing with state-of-the-art solutions.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (NSFC) under Grant 62372426, the Youth Innovation Promotion Association of the Chinese Academy of Science under Grant 2023481, Basic Research Program of Jiangsu under Grant BK20250124, and the Fundamental Research Funds for the Central Universities under Grant WK2150250043.

References

- [1] Abdelrahman Abuarqoub. 2020. A review of the control plane scalability approaches in software defined networking. *Future Internet* 12, 3 (2020), 49.
- [2] Sujoy Acharya. 2018. *Apache Ignite Quick Start Guide: Distributed data caching and processing made easy*. Packt Publishing Ltd.
- [3] Bartosz Baliś, Andrzej Broński, and Mateusz Szarek. 2022. Auto-scaling of Scientific Workflows in Kubernetes. In *International Conference on Computational Science*. Springer, 33–40.
- [4] Tim Bell, B Bompastor, S Bukowiec, J Castro Leon, MK Denis, J van Eldik, M Fermín Lobo, L Fernandez Alvarez, D Fernandez Rodriguez, A Marino, et al. 2015. Scaling the CERN OpenStack cloud. In *Journal of Physics: Conference Series*, Vol. 664. IOP Publishing, 022003.
- [5] Brendan Burns, Joe Beda, Kelsey Hightower, and Lachlan Evenson. 2022. *Kubernetes: up and running*. " O'Reilly Media, Inc".
- [6] S Cash, V Jain, L Jiang, Alexei Karve, J Kidambi, M Lyons, T Mathews, S Mullen, M Mulsow, and N Patel. 2016. Managed infrastructure with ibm cloud openstack services. *IBM Journal of Research and Development* 60, 2-3 (2016), 6–1.
- [7] Joris Claassen, Ralph Koning, and Paola Grosso. 2016. Linux containers networking: Performance and scalability of kernel modules. In *NOMS 2016-2016 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 713–717.
- [8] Constantin-Cosmin Crecana and Florin Pop. 2018. Monitoring-based auto-scalability across hybrid clouds. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing*. 1087–1094.
- [9] Edward Curry, Desmond Chambers, and Gerard Lyons. 2004. Extending message-oriented middleware using interception. In *Third International Workshop on Distributed Event-Based Systems (DEBS'04)*, ICSE, Vol. 4. IET.
- [10] Ludwik Czaja. 2018. Remote procedure call. In *Introduction to distributed computer systems*. Springer, 141–155.
- [11] Michael Dalton, David Schultz, Jacob Adriaens, Ahsan Arefin, Anshuman Gupta, Brian Fahs, Dima Rubinstein, Enrique Cauch Zermeno, Erik Rubow, James Alexander Docauer, et al. 2018. Andromeda: Performance, isolation, and velocity at scale in cloud network virtualization. In *15th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI})* 18, 373–387.
- [12] Milena Talavera Dave Brown. 2025. Leaping ahead: The power of cloud network innovation. https://d1.awsstatic.com/events/Summits/reinvent2022/NET211-L_Leaping-ahead-The-power-of-cloud-network-innovation.pdf
- [13] James Denton. 2015. *Learning OpenStack Networking (Neutron)*. Packt Publishing Ltd.
- [14] Philippe Dobbelaere and Kyumars Sheykh Esmaili. 2017. Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations: Industry Paper. In *Proceedings of the 11th ACM international conference on distributed and event-based systems*. 227–238.
- [15] Andrew D Ferguson, Steve Gribble, Chi-Yao Hong, Charles Killian, Waqar Mohsin, Henrik Muehe, Joon Ong, Leon Poutievski, Arjun Singh, Lorenzo Vicisano, et al. 2021. Orion: Google's {Software-Defined} Networking Control Plane. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 83–98.
- [16] OpenInfra Foundation. 2025. Get Started with an OpenStack Public Cloud. <https://www.openstack.org/marketplace/public-clouds>
- [17] Edgar Gabriel, Graham E Fagg, George Bosilca, Thara Angskun, Jack J Dongarra, Jeffrey M Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, et al. 2004. Open MPI: Goals, concept, and design of a next generation MPI implementation. In *European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting*. Springer, 97–104.
- [18] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan MG Wassel, Zhehua Wu, Sunghwan Yoo, et al. 2022. Aquila: A unified, low-latency fabric for datacenter networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 1249–1266.
- [19] Zhiqiang He, Dongyang Wang, Binzhang Fu, Kun Tan, Bei Hua, Zhi-Li Zhang, and Kai Zheng. 2020. MASQ: RDMA for virtual private cloud. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 1–14.
- [20] Bhole Rahul Hiran et al. 2018. A study of apache kafka in big data stream processing. In *2018 International Conference on Information, Communication, Engineering and Technology (ICICET)*. IEEE, 1–3.
- [21] Mahmoud Imdoukh, Intiaz Ahmad, and Mohammad Gh Alfaiakawi. 2020. Machine learning-based auto-scaling for containerized applications. *Neural Computing and Applications* 32 (2020), 9745–9760.
- [22] Kasun Indrasiri and Danesh Kuruppu. 2020. *gRPC: up and running: building cloud native applications with Go and Java for Docker and Kubernetes*. O'Reilly Media.
- [23] Keon Jang, Justine Sherry, Hitesh Ballani, and Toby Moncaster. 2015. Silo: Predictable message latency in the cloud. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 435–448.
- [24] Anuj Kalia, Michael Kaminsky, and David G Andersen. 2016. FaST: Fast, Scalable and Simple Distributed Transactions with Two-Sided (RDMA) Datagram RPCs.. In *OSDI*, Vol. 16. 185–201.
- [25] Anuj Kalia, Michael Kaminsky, and David G Andersen. 2019. Datacenter RPCs can be general and fast. In *Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation*. 1–16.
- [26] David G Kendall. 1953. Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain. *The Annals of Mathematical Statistics* (1953), 338–354.
- [27] Teemu Koponen, Martin Casado, Natasha Gude, Jeremy Stribling, Leon Poutievski, Min Zhu, Rajiv Ramanathan, Yuichiro Iwata, Hiroaki Inoue, Takayuki Hama, et al. 2010. Onix: A distributed control platform for large-scale production networks.. In *OSDI*, Vol. 10. 6.
- [28] Giacomo Lanciano, Filippo Galli, Tommaso Cucinotta, Davide Bacciu, and Andrea Passarella. 2021. Predictive auto-scaling with OpenStack Monasca. In *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing*. 1–10.
- [29] Bob Lantz and Brian O'Connor. 2015. A mininet-based virtual testbed for distributed SDN development. *ACM SIGCOMM Computer Communication Review* 45, 4 (2015), 365–366.
- [30] Jean-Yves Le Boudec. 2010. *Performance evaluation of computer and communication systems*. Vol. 2. Epfl Press Lausanne.
- [31] Bojie Li, Gefei Zuo, Wei Bai, and Lintao Zhang. 2021. 1Pipe: scalable total order communication in data center networks. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 78–92.
- [32] Stanley Lima, Alvaro Rocha, and Licinio Roque. 2019. An overview of OpenStack architecture: a message queuing services node. *Cluster Computing* 22, 3 (2019), 7087–7098.
- [33] Tania Lorido-Botran, Jose Miguel-Alonso, and Jose A Lozano. 2014. A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of grid computing* 12 (2014), 559–592.
- [34] Stuart Mitchell, Michael OSullivan, and Iain Dunning. 2011. PuLP: a linear programming toolkit for python. *The University of Auckland, Auckland, New Zealand* 65 (2011).
- [35] Habib Mostafaei, Gabriele Lospoto, Roberto Di Lallo, Massimo Rimondini, and Giuseppe Di Battista. 2020. A framework for multi-provider virtual private networks in software-defined federated networks. *International Journal of Network Management* 30, 6 (2020), e2116.
- [36] Francisco Daniel Muñoz Escó. 2021. ZeroMQ: PUSH-PULL pattern. (2021).
- [37] Juuso Nuikka. 2021. *Comparison of Cloud Native messaging technologies*. Master's thesis.
- [38] Tian Pan, Nianbing Yu, Chenhao Jia, Jianwen Pi, Liang Xu, Yisong Qiao, Zhiguo Li, Kun Liu, Jie Lu, Jianyuan Lu, et al. 2021. Sailfish: Accelerating cloud-scale multi-tenant multi-service gateways with programmable switches. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 194–206.
- [39] Helge Parzyjegl, Christian Wernecke, Gero Mühl, Eike Schweissguth, and Dirk Timmermann. 2019. Implementing content-based publish/subscribe with OpenFlow. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*. 1392–1395.
- [40] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Joe Stringer, Pravin Shelar, et al. 2015. The Design and Implementation of Open {vSwitch}. In *12th USENIX symposium on networked systems design and implementation (NSDI 15)*. 117–130.
- [41] Paul Rad, Rajendra V Boppana, Palden Lama, Gilad Berman, and Mo Jamshidi. 2015. Low-latency software defined network for high performance clouds. In *2015 10th System of Systems Engineering Conference (SoSE)*. IEEE, 486–491.
- [42] Francisco Romero, Gohar Irfan Chaudhry, go Goiri, Pragna Gopa, Paul Batum, Neeraja J Yadwadkar, Rodrigo Fonseca, Christos Kozyrakis, and Ricardo Bianchini. 2021. FaaS: A transparent auto-scaling cache for serverless applications. In *Proceedings of the ACM Symposium on Cloud Computing*. 122–137.
- [43] Gabriele Russo Russo, Antonio Schiazza, and Valeria Cardellini. 2021. Elastic Pulsar Functions for Distributed Stream Processing. In *Companion of the ACM/SPEC International Conference on Performance Engineering*. 9–16.
- [44] Omar Sefraoui, Mohammed Aissaoui, Mohsine Eleuldj, et al. 2012. OpenStack: toward an open-source solution for cloud computing. *International Journal of Computer Applications* 55, 3 (2012), 38–42.
- [45] S Seleznev and V Yakovlev. 2019. Industrial Application Architecture IoT and protocols AMQP, MQTT, JMS, REST, CoAP, XMPP, DDS. *International Journal of Open Information Technologies* 7, 5 (2019), 17–28.
- [46] Amazon Web Services. 2025. Step and simple scaling policies for Amazon EC2 Auto Scaling - Amazon EC2 Auto Scaling. <https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-simple-step.html#policy-creating-policy-aws-cli>
- [47] Rahul Sharma and Mohammad Atyab. 2022. Introduction to Apache Pulsar. In *Cloud-Native Microservices with Apache Pulsar*. Springer, 1–22.
- [48] Mitesh Soni. 2018. *Practical AWS Networking: Build and Manage Complex Networks Using Services Such as Amazon VPC, Elastic Load Balancing, Direct Connect, and Amazon Route 53*. Packt Publishing Ltd.
- [49] OASIS Standard. 2012. Oasis advanced message queuing protocol (amqp) version 1.0. *International Journal of Aerospace Engineering Hindawi www. hindawi. com* (2012).

- [50] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. {EPaxos} Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 613–632.
- [51] Chengkun Wei, Xing Li, Ye Yang, Xiaochong Jiang, Tianyu Xu, Bowen Yang, Taotao Wu, Chao Xu, Yilong Lv, Haifeng Gao, et al. 2023. Achelous: Enabling Programmability, Elasticity, and Reliability in Hyperscale Cloud Networks. In *Proceedings of the ACM SIGCOMM 2023 Conference*, 769–782.
- [52] John Wilkes. 2020. Yet more Google compute cluster trace data. Google research blog. Posted at <https://ai.googleblog.com/2020/04/yet-more-google-compute-cluster-trace.html>.
- [53] Jiang Yongguo, Liu Qiang, Qin Changshuai, Su Jian, and Liu Qianqian. 2019. Message-oriented middleware: A review. In *2019 5th International Conference on Big Data Computing and Communications (BIGCOM)*. IEEE, 88–97.
- [54] Ahmed Dalhatu Yusuf, Saleh Abdullahi, Moussa Mahamat Boukar, and Salisu Ibrahim Yusuf. 2021. Collision Resolution Techniques in Hash Table: A Review. *International Journal of Advanced Computer Science and Applications* 21, 9 (2021).
- [55] Ahmed Dalhatu Yusuf, Saleh Abdullahi, Moussa Mahamat Boukar, and Salisu Ibrahim Yusuf. 2021. A Multi-Indexes Based Technique for Resolving Collision in a Hash Table. *International Journal of Computer Science & Network Security* 21, 9 (2021), 339–345.

A Thresholds Estimation Method

In Section 4, we have defined two thresholds τ_P and τ_N , which are set artificially based on experience. However, a better approach would be to set them according to the current system performance and a desired end-to-end latency. To this end, for each time period T (e.g., 10 minutes [21, 33]), we present a method to estimate thresholds for next time period.

Considering that the control plane overhead mainly comes from the RPC connections, we mainly model the message delivery latency performance of the RPC path, and abstract the RPC path with queueing theory. We denote $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ as the VPC set, where $|\mathcal{V}|$ is the number of VPCs in the cloud. P_j denotes the set of ports and N_j denotes the set of compute nodes associated with VPC $v_j \in \mathcal{V}$. According to our switching mechanism, the set of VPCs using the RPC path can be expressed as

$$\mathcal{V}' = \{v_j | |P_j| < \tau_P, v_j \in \mathcal{V}\} \cap \{v_j | |N_j| < \tau_N, v_j \in \mathcal{V}\} \quad (1)$$

Our queuing model is described by Kendall's notation [26], i.e., $A/B/n$. Specifically, A denotes the Poisson incoming message distribution, which characterized by a parameter λ .

$$P_A(x) = \frac{e^{-\lambda} \lambda^x}{x!}, x = 0, 1, 2, \dots \quad (2)$$

λ is the average arrival rate of control messages in period T depending on the total VPC ports configured through the RPC path $\sum_{v_j \in \mathcal{V}'} |P_j|$. With a desired delivery latency μ , the Poisson delivery distribution B is characterized by the expected delivery rate $\frac{1}{\mu}$. Moreover, n denotes the number of RPC connections, i.e., the total VPC nodes $|\bigcup_{v_j \in \mathcal{V}'} N_j|$. Since RPC connections bring overhead to the resource-limited controller, the number of RPC connections is limited. That is, $n < \psi$ where ψ denotes the maximum number of connections that a controller can carry. In other words, threshold τ_N should be also less than ψ . The end-to-end latency is the sum of the waiting time ω and the delivery latency μ . According to the previous work [30], the wait time ω can be expressed as

$$\omega = \frac{\gamma (\lambda \mu)^n \rho}{n! (1 - \rho)^2} \quad (3)$$

where $\gamma = 1 / \left[\sum_{x=0}^{n-1} \frac{(n\rho)^x}{x!} + \frac{(n\rho)^n}{n!(1-\rho)} \right]$ and $\rho = \frac{\lambda \mu}{n}$.

Therefore, for a desired end-to-end latency $\omega + \frac{1}{\mu}$, τ_P and τ_N can be estimated by solving the above optimization problem using the solver (e.g., PuLP [34]).

B VPC Assignment Algorithm

This section describes the VPC assignment algorithm.

Message Queue Model. We use $\mathcal{Q} = \{q_1, q_2, \dots, q_{|\mathcal{Q}|}\}$ to denote the queues in the MQ server, where $|\mathcal{Q}|$ is the number of queues. For each queue q_i , the hash map capacity of the queue's filter is denoted as C and the remaining hash capacity is denoted as C_i . We denote $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ as the VPC set, where $|\mathcal{V}|$ is the number of VPCs in the cloud. N_j denotes the compute nodes associated with VPC v_j . To implement redundancy-free unicast message delivery, VPC v_j needs to occupy $|N_j| + 1$ hash addresses, $|N_j|$ for unicast and one for multicast.

Problem Formalization. For sake of successful message delivery, we consider the following two facts or constraints. 1) each VPC will subscribe to exactly one topic. 2) The number of message tags in a queue should not exceed the hash capacity. Specifically, for a queue, the required number of tags is the total number of compute nodes associated with these VPCs. For example, when VPCs v_m and v_n are assigned to queue q_k , $|N_m| + |N_n| + 2$ tags are required in the queue, that is, $|N_m| + |N_n| + 2$ hash addresses in the filter. The objective is to minimize the number of occupied queues, so as to pursue better latency performance for MQ path. We give problem formalization as follows:

$$\begin{aligned} & \min \sum_{q_i \in \mathcal{Q}} y_i \\ \text{s.t.} & \begin{cases} \sum_{q_i \in \mathcal{Q}} x_i^j = 1, & \forall v_j \in \mathcal{V} \\ \sum_{v_j \in \mathcal{V}} (|N_j| + 1) x_i^j < C, & \forall q_i \in \mathcal{Q}, v_j \in \mathcal{V} \\ y_i \geq x_i^j, & \forall q_i \in \mathcal{Q}, v_j \in \mathcal{V} \\ x_i^j \in \{0, 1\}, & \forall q_i \in \mathcal{Q}, v_j \in \mathcal{V} \\ y_i \in \{0, 1\}, & \forall q_i \in \mathcal{Q} \end{cases} \quad (4) \end{aligned}$$

where a binary $x_i^j \in \{0, 1\}$ denotes whether a VPC $v_j \in \mathcal{V}$ is assigned to queue $q_i \in \mathcal{Q}$ or not, and binary $y_i \in \{0, 1\}$ represents whether queue $q_i \in \mathcal{Q}$ is assigned to VPC(s) or not. The first set of equations means that each VPC should be assigned to one and only one queue. The second set of inequalities means that the required number of tags in each queue should not exceed the hash capacity of queue's filter. The third set of inequalities means that a queue is occupied if at least one VPC is subscribed to this queue.

Algorithm 1 VPC Assignment

- 1: **INPUT:** VPC v_j with $|N_j|$ compute nodes.
 - 2: Calculate the remaining hash capacity C_i of existing queues $q_i \in \{y_i = 1 | q_i \in \mathcal{Q}\}$;
 - 3: Find the queue q_k with minimum C_k ;
 - 4: **if** $q_k \neq \text{NULL}$ **then**
 - 5: Assign VPC v_j to queue q_k ;
 - 6: **else**
 - 7: Create a new queue q' and assign VPC v_j to q' .
 - 8: **end if**
-

Algorithm Description. We design an online algorithm to assign topics for new VPCs. The assignment process is described in Algorithm 1. When a VPC v_j expects to choose MQ for control message delivery, we calculate the remaining hash capacity of each

queue filter and find the queues that meet the tag requirements (Lines 2-3). Then, we choose the queue with the smallest remaining hash capacity, denote as q_k , and assign this VPC to the queue (Line 5). Otherwise, a new queue will be created for this VPC (Line 7).